

CODE 1:

```
praks@praks:~$ nano fcfs.c
praks@praks:~$ gcc fcfs.c -o fcfs
./fcfs
Enter number of processes: 3
Enter burst time for each process:
P1: 5
P2: 2
P3: 3

Process BT      WT      TAT
P1      5        0        5
P2      2        5        7
P3      3        7       10
praks@praks:~$ |
```

```
praks@praks: ~ × + ∨
GNU nano 8.7.1 fcfs.c *
#include <stdio.h>

int main() {
    int n, i;
    int bt[20], wt[20], tat[20];

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time for each process:\n");
    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
    }

    // Waiting time
    wt[0] = 0;
    for (i = 1; i < n; i++) {
        wt[i] = wt[i - 1] + bt[i - 1];
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBT\tWT\tTAT\n");
    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
    }

    return 0;
}|
```

CODE 2:

```
praks@praks: ~  
GNU nano 8.7.1 priority.c *  
#include <stdio.h>  
  
int main() {  
    int n, i, j, temp;  
    int bt[20], priority[20], wt[20], tat[20];  
  
    printf("Enter number of processes: ");  
    scanf("%d", &n);  
  
    printf("Enter burst time and priority:\n");  
    for (i = 0; i < n; i++) {  
        printf("P%d Burst Time: ", i + 1);  
        scanf("%d", &bt[i]);  
        printf("P%d Priority: ", i + 1);  
        scanf("%d", &priority[i]);  
    }  
  
    // Sort according to priority  
    for (i = 0; i < n - 1; i++) {  
        for (j = i + 1; j < n; j++) {  
            if (priority[i] > priority[j]) {  
                temp = priority[i];  
                priority[i] = priority[j];  
                priority[j] = temp;  
  
                temp = bt[i];  
                bt[i] = bt[j];  
                bt[j] = temp;  
            }  
        }  
    }  
  
    // Waiting time  
    wt[0] = 0;  
    for (i = 1; i < n; i++) {  
        wt[i] = wt[i - 1] + bt[i - 1];  
    }  
  
    // Turnaround time  
    for (i = 0; i < n; i++) {  
        tat[i] = wt[i] + bt[i];  
    }  
  
    printf("\nProcess\tBT\tPriority\tWT\tTAT\n");  
    for (i = 0; i < n; i++) {  
        printf("P%d\t%d\t%d\t%d\t%d\n", i + 1, bt[i], priority[i], wt[i], tat[i]);  
    }  
  
    return 0;  
}
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute	^C Location	M-U Undo
^X Exit	^R Read File	^N Replace	^U Paste	^J Justify	^_ Go To Line	M-E Redo

```
praks@praks:~$ nano priority.c
praks@praks:~$ gcc priority.c -o priority
./priority
Enter number of processes: 5
Enter burst time and priority:
P1 Burst Time: 2
P1 Priority: 8
P2 Burst Time: 7
P2 Priority: 6
P3 Burst Time: 4
P3 Priority: 2
P4 Burst Time: 5
P4 Priority: 9
P5 Burst Time: 6
P5 Priority: 4

Process BT      Priority      WT      TAT
P1      4      2      0      4
P2      6      4      4      10
P3      7      6      10     17
P4      2      8      17     19
P5      5      9      19     24
praks@praks:~$ |
```

CODE 3:

```
praks@praks:~$ nano roundrobin.c
praks@praks:~$ gcc roundrobin.c -o roundrobin
./roundrobin
Enter number of processes: 2
Enter burst time:
P1:
1
P2: 2
Enter time quantum: 6

Process BT      WT      TAT
P1      1        0        1
P2      2        1        3
praks@praks:~$ |
```

praks@praks: ~

GNU nano 8.7.1roundrobin.c *

```
#include <stdio.h>

int main() {
    int n, i;
    int bt[20], rem_bt[20];
    int wt[20], tat[20];
    int quantum;
    int time = 0;
    int done;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter burst time:\n");
    for (i = 0; i < n; i++) {
        printf("P%d: ", i + 1);
        scanf("%d", &bt[i]);
        rem_bt[i] = bt[i];
        wt[i] = 0;
    }

    printf("Enter time quantum: ");
    scanf("%d", &quantum);

    // Round Robin
    while (1) {
        done = 1;
        for (i = 0; i < n; i++) {
            if (rem_bt[i] > 0) {
                done = 0;
                if (rem_bt[i] > quantum) {
                    time = time + quantum;
                    rem_bt[i] = rem_bt[i] - quantum;
                } else {
                    time = time + rem_bt[i];
                    wt[i] = time - bt[i];
                    rem_bt[i] = 0;
                }
            }
        }
        if (done == 1)
            break;
    }

    // Turnaround time
    for (i = 0; i < n; i++) {
        tat[i] = wt[i] + bt[i];
    }

    printf("\nProcess\tBT\tWT\tTAT\n");
    for (i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\n", i + 1, bt[i], wt[i], tat[i]);
    }

    return 0;
}
```

File to insert [from ./]:

^G Help

^C Cancel

M-F New Buffer

M-N No Conversion

^X Execute Command

^T Browse